

CLAUDIA MARIA LOPEZ BOMBIN

LUCÍA LOZANO ISAC

SEPTIEMBRE, 2025

PRÁCTICA 1

O SEA, ¿CÓMO QUE $2 + 2 \equiv 5 \pmod{1}$?

MATEMÁTICA DISCRETA

DAVID ALFAYA SÁNCHEZ

GRUPO GB06

ÍNDICE

INTRODUCCIÓN	3
DISEÑO E IMPLEMENTACIÓN.....	4
MANUAL DE USUARIO	27
CONCLUSIONES	29
AUTORÍA.....	31
BIBLIOGRAFÍA.....	32

INTRODUCCIÓN

Este documento, realizado por Claudia Maria Lopez Bombin y Lucía Lozano Isac – alumnas de 2B del Grado en Ingeniería Matemática e Inteligencia Artificial en la Universidad Pontificia Comillas-, corresponde a la memoria complementaria a la primera practica de la asignatura Matemática Discreta, encuadrada en las nociones de matemática modular estudiadas.

El término “matemática modular” fue introducido en 1801 por Carl Friedrich Gauss en su libro *Disquisitiones Arithmeticae*. Se define como “el conjunto de métodos que derivan del estudio del resto obtenido mediante una división euclídea” (Gauss, 1801). Aunque pueda parecer un término bastante abstracto y complejo, la matemática modular es un concepto que usamos en nuestras vidas diarias, casi sin apenas darnos cuenta. Desde leer la hora en un reloj analógico a organizar los días de la semana, pasando por escuchar ciclos musicales, la aritmética modular se cuela en nuestro día a día de la manera más insospechada.

En este contexto se encuadra esta práctica, cuyo objetivo principal es la aplicación de los conceptos de modularidad, estudiados y demostrados en clase, para resolver problemas de aritmética y afianzar los conceptos de esta área fundamental de las Matemáticas.

Como objetivos secundarios de esta práctica encontramos:

- Profundizar en el desarrollo de código en el entorno de programación de Python, creando una librería de funciones para el cálculo de conceptos matemáticos avanzados,
- Implementar algoritmos eficientes para resolver problemas de matemática modular,
- Aprender a programar una interfaz de usuario y manejar entradas por terminal,
- Analizar y manejar los distintos errores que se puedan producir a la hora de ejecutar el código y al realizar operaciones,
- Validar el correcto funcionamiento del código implementado mediante el uso de archivos tests y de la librería *pytest*,
- Fomentar el trabajo colaborativo.

El proyecto consiste en la implementación de un sistema integral con dos archivos de tipo .py:

1. **modular.py**: un archivo que implementa las definiciones de las funciones que se encargan de resolver cada una de las operaciones modulares que posteriormente el usuario ejecuta.
2. **imatlab.py**: archivo que incluye el código necesario para entender las entradas del usuario y redirigirla a las funciones adecuadas de modular.py

DISEÑO E IMPLEMENTACIÓN

1. Arquitectura del código

La arquitectura del código de este proyecto es de tipo modular, con capas muy bien definidas, para la compartimentalización de las lógicas de los módulos para así facilitar el mantenimiento, el aislamiento de errores y la compartimentalización de estos (así el cambio en una de las funciones de no afecta a la ejecución del resto) . También ayuda en el proceso de escalabilidad y optimización individualizada de cada una de las funciones.

El sistema se divide en dos módulos principales que interactúan entre sí:

a. modular.py: el núcleo computacional

El programa modular.py se comporta como el cerebro matemático del sistema, usando algoritmos optimizados para resolver las operaciones solicitadas. Las funciones se dividen en cuatro niveles, dependiendo de su complejidad:

- Operaciones fundamentales :

Incluye algoritmos básicos de teoría de números, con modificaciones claras para reducir el número de operaciones internas y, por lo tanto, su tiempo de ejecución. Un ejemplo claro de este nivel es la función `es_primo(n)`, que utiliza propiedades matemáticas y evita la comprobación de números pequeños, ya que resultan redundantes.

- Operaciones avanzadas:

Ejemplos claros de este nivel son las funciones `bezout(n, m)`, que implementa recursivamente la identidad de Bézout y calculando no solo el máximo común divisor sino además los coeficientes que satisfacen la ecuación diofántica solicitada, y la función `potencia_mod_p(base, exp, p)`, que utiliza el método de duplicación y cuadrados sucesivos para reducir la complejidad del cálculo de la potencia modular.

- Aritmética modular especializada

Incluye las implementaciones de conceptos más avanzados como `legendre(n, p)`, que contiene optimizaciones específicas para el trabajo con números primos. Están también incluidos algoritmos básicos para empezar a resolver ecuaciones cuadráticas modulares simples.

- Sistemas de ecuaciones modulares

Funciones que se basan en el Teorema “Chino del Resto” para resolver ecuaciones modulares más complejas, ya que acepta el manejo de ecuaciones con módulos no coprimos. Incluye también verificaciones de compatibilidades y un manejo más robusto de las entradas.

b. Imatlab.py: la capa de interfaz

El archivo `imatlab.py` funciona como puente entre el usuario y las funciones del núcleo `modular.py`, permitiendo la ejecución de los comandos de manera interactiva o por entradas de texto por terminal.

La arquitectura de este código se basa en patrones de comandos, simplificando el lenguaje para su ejecución, basándose en las siguientes funcionalidades de manejo:

- Sistema de parsing inteligente

Implementado fundamentalmente en la función `parse_args(args, comando)`. Encontramos un analizador sintáctico que es capaz de interpretar los distintos formatos de entrada de comandos, desde argumentos simples hasta listas anidadas. Detecta de manera segura la estructura de los parámetros y los adapta a los requisitos de entrada de la función matemática a la que deban ser redirigidos.

- Gestión dinámica de los comandos

Sirviéndose de un diccionario de despacho, el programa es capaz de asociar el comando de usuario con la función matemática correspondiente del módulo `modular.py`. El diseño que presenta permite extender las funciones de manera muy sencilla: tan solo requiere actualizar el diccionario para las nuevas funciones.

- Manejo de errores

Implementa una estrategia eficiente para gestionar las excepciones para transformar los errores en mensajes comprensibles para el usuario (“NE” para No Existe y “NOP” para comando no reconocible u operación no disponible).

El procesamiento de los comandos sigue una secuencia definida por:

1. Recepción y validación: verifica la estructura del comando y comprueba el patrón “nombre(argumento)”.
2. Tokens y análisis: separamos el comando a ejecutar de los argumentos introducidos.
3. Enrutamiento de comandos: Determina la función matemática designada por el comando y aplica las transformaciones de los argumentos necesarias para la correcta ejecución de las funciones.
4. Control de errores: llamamos a la función con un mecanismo que captura las excepciones lanzadas, diferenciando entre errores matemáticos y errores del sistema.
5. Formateo de salida: formatea los outputs para adecuarse a las reglas de presentación específica de cada función.

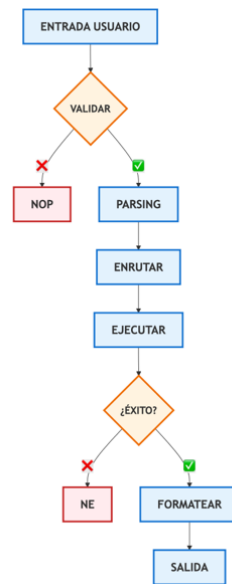


Figura 1. Diagrama de flujo de la secuencia de procesamiento de comandos. Elaboración propia.

El diseño del código incluye técnicas de optimización algorítmicas y de tipo arquitectónico para proporcionar una base sólida y robusta sobre la cual sustentar el sistema IMAT-LAB y así equilibrar el rendimiento, mantenimiento y facilidad de uso del sistema:

- Algoritmos optimizados: implementamos mejoras en base a teoremas claros y eficientes.
- Detección precoz: detectamos condiciones terminales y evitamos continuar con cálculos innecesarios.
- Principios de Memoización (memoize): intentamos aplicar algunos principios de memoización (entendido como la “técnica de optimización de programación en la cual se almacenan los resultados de funciones que requieren cálculos costosos” (Fenollosa, 2023)) para reutilizar resultados intermedios.
- Manejo eficiente de la memoria: usamos estructuras de datos adecuadas para evitar la sobrecarga de la memoria.

2. Algoritmos principales

El Core del sistema se encuentra en la implementación eficiente de algoritmos de teoremas fundamentales de la teoría de números y la aritmética modular. Las funcionalidades están programadas centrándose en profundizar en su base matemática, explorando teoremas para reducir su complejidad y tiempos de ejecución, así como pensando en una rápida aplicabilidad en la práctica de problemas computacionales complejos y reales.

En este apartado analizaremos en profundidad cada uno de los algoritmos usado en la programación de las funciones, justificando las decisiones de diseño y el flujo del programa en su ejecución.

- `es_primo(n:int) -> bool:`

El algoritmo en el cual se basa esta función es una optimización del teorema básico de verificar la divisibilidad por todos los números menores de $\sqrt{n}$. Nos basamos en una propiedad matemática que establece que todos los números primos mayores que 3 siguen la forma $6k \pm 1$ (Garcia-Pelayo, 2012). Esta observación nos permite reducir el número de comprobaciones necesarias, solo teniendo que comprobar si cumplen esta expresión.

Comenzamos manejando números pequeños, eliminando los números menores que 1 (ya que 1, 0 y los números negativos no se consideran primos) y los múltiplos de 2, 3, 5 y 7 (los primeros números primos). Esto hace que se eliminen la mayoría de los números no primos. Posteriormente, comprobamos si los números enteros antes de la raíz son divisores, solo teniendo que comprobar un mínimo número de divisores.

Hemos decidido implementar el algoritmo de manera iterativa, frente a una recursiva, para tener en cuenta una mayor eficiencia de la memoria y para no perder tiempo en la ejecución, teniendo que llamar a la función otra vez.

Destacamos que este algoritmo es especialmente eficaz para la verificación de números de tamaño pequeño y moderado, pudiendo llegar a tardar más a la hora de verificar números más grandes.

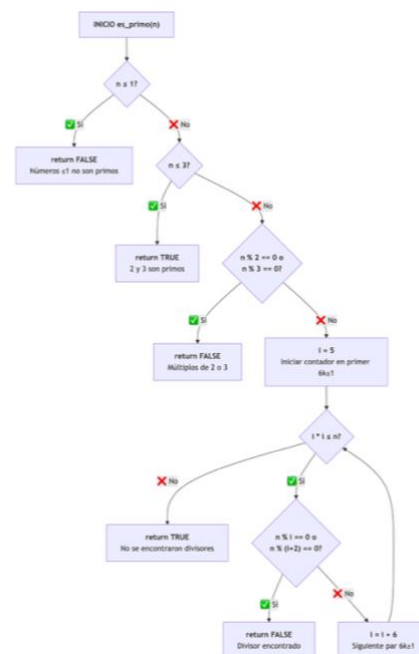


Figura 2. Diagrama de flujo de la función `es_primo(n:int) -> bool`. Elaboración propia.

El algoritmo comienza realizando verificaciones rápidas para casos pequeños y múltiplos de 2 y 3. Luego procesa a verificar divisores potenciales en incrementos de seis, teniendo en cuenta la forma de los números primos, explicado previamente, reduciendo significativamente el número de divisiones necesarias.

- `lista_primos(a:int,b:int)->list:`

El algoritmo se basa en encontrar una lista de primos dado un intervalo inicial, tal y como se describen en los argumentos de la función. La función recibe un entero a que denotaría el número desde el que partimos y otro entero b que denotaría el límite del análisis. Haciendo uso de la función `es_primo(n)` de la que se ha hablado en el apartado anterior vamos recorriendo con un `for` los números que se encuentran en el intervalo establecido, analizando si son o no primos. En caso de que lo sean, se añaden a la lista `primos`. Finalmente, una vez analizados todos los números, devolvemos la lista con aquellos que sean primos.

El algoritmo, al hacer uso de la función `es_primo(n)` está basado en el teorema de la verificación la divisibilidad de todos los números menores que $\sqrt{n}$. Por ello, la función `es_primo(n)` nos devuelve directamente si ese número es primo o no, sin necesidad de volverla a implementar.

Este algoritmo se ha implementado de manera iterativa, por tanto, funciona para cualquier intervalo que introduzca el usuario. Con un intervalo amplio el funcionamiento no se ve afectado ya que se ha realizado de manera eficiente, y como hace uso de la función `es_primo(n)` y se ha comprobado antes y mediante los tests, que es eficiente para números altos, por consecuencia la función `lista_primos()` también es eficiente.

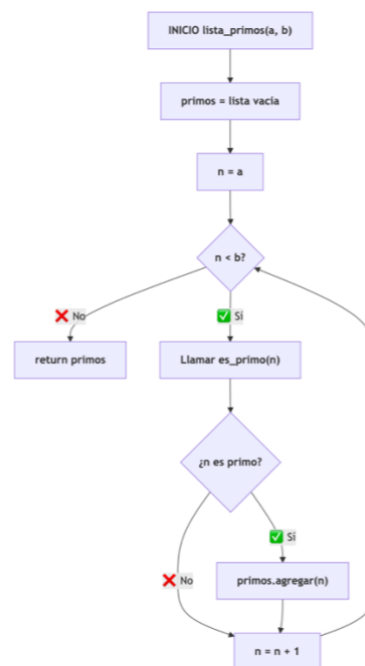


Figura 3. Diagrama de flujo de la función `lista_primos (a:int,b:int) -> list`. Elaboración propia.

El diagrama de flujo deja constancia del bucle principal, donde vemos cómo se van evaluando uno a uno los números dentro del intervalo (a,b) . A su vez se lleva a cabo la comprobación mediante la función `es_primo` de si son o no primos. En caso de que lo sean se añaden a la lista de primos, si no, no. Si no hay más elementos se devuelve la lista, si hay, seguimos iterando.

- factorizar (n:int) -> Dict[int, int]:

Esta función se basa en el Teorema Fundamental de la Aritmética, - enunciado primeramente por Euclides hace más de 2000 años, formulado definitivamente por Gauss y profundamente estudiado por los matemáticos Hardy y Wright-, que establece que todo número entero mayor que 1 puede descomponerse de forma única como producto de potencias de números primos (Hardy & Wright, 1938).

La implementación comienza manejando y comprobando casos especiales: el número 0, que por definición no tiene factorización única en números primos, y los números negativos, para los cuales trabajamos con su valor absoluto para mantener la consistencia matemática. Luego eliminamos sistemáticamente todos los factores de 2 mediante divisiones sucesivas (guardando el número de divisiones para los exponentes), lo que simplifica el número para factorizar en base a números primos impares.

Hemos implementado el algoritmo de manera iterativa con tres fases diferenciadas. La primera es la factorización exclusiva en base al número 2, la segunda es la factorización de números impares hasta la raíz y la última, el número primo restante (en caso de hallarse).

Destacamos que este algoritmo es especialmente eficiente para números no primos con factores pequeños, ya que los elimina en las primeras iteraciones. Para números más grandes, sigue manteniendo la misma complejidad, pero aun con tiempos de ejecución bajos.

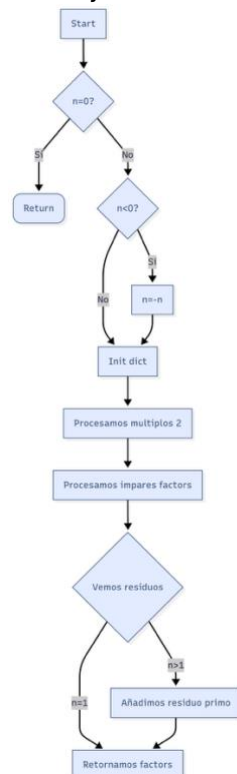


Figura 4. Diagrama de flujo de la función factorizar (n:int) -> Dict[int, int]. Elaboración propia.

El diagrama de flujo evidencia la estructura trifásica explicada previamente. Refleja además las optimizaciones clave: tratamiento de pares, verificación de factores menores que la raíz y adición de los factores primos restantes.

- `mcd(a:int,b:int)->int:`

Este algoritmo busca hallar el máximo común divisor entre dos números enteros dados. Para que el proceso sea más sencillo, primero convertimos los números en positivo en caso de que fueran negativos. Basándonos en la teoría vista en clase, sabemos que hay dos tipos de resultados; o bien a y b son coprimos, por tanto, su máximo común divisor es 1 o bien hay algún divisor común en a y b distinto de 1.

Aplicamos el Teorema de Euclides o Algoritmo de Euclides que se basa en la propiedad de que el máximo común divisor de dos números no cambia si se divide el número mayor entre el número menor y se reemplaza cada número por el divisor y el resto. Haciendo uso de la propiedad si a distinto de 0, calculamos el resto de b módulo a y cambiamos (a,b) por (r,a) . De esta manera, acabamos obteniendo como resultado 1 u otro número en caso de que no sean primos relativos.

La implementación del algoritmo es la siguiente; definimos un bucle `while` que se ejecutará siempre y cuando b sea distinto de 0. Si entramos en el bucle, cambiamos (a,b) por (b,r) , siendo r el resto que sale de calcular el resto de dividir a entre b que funcionará tanto si a es mayor que b como si b es mayor que a . Realizada esta iteración, comprobamos si b es 0 o no. En caso de que no lo sea, seguimos entrando en el bucle, en caso de que sí, devolvemos a , ya que será a la variable que contendrá el máximo común divisor de a y b .

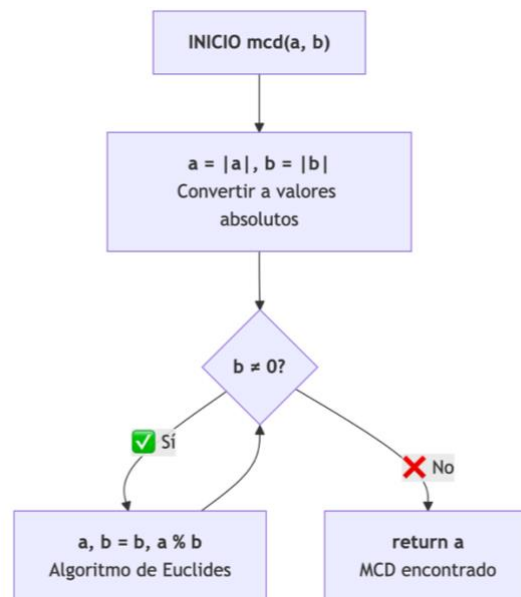


Figura 5. Diagrama de flujo de la función `mcd(a:int,b:int)->int`. Elaboración propia.

El diagrama de flujo deja constancia de la transformación inicial de a y b en positivos. Mediante un bucle con condición b distinto de 0, vamos iterando, intercambiando los valores de a y b y ajustando el valor de b en cada iteración aplicando los teoremas mencionados. Si b vale 0, se retorna a .

- `bezout(n:int,m:int)->tuple[int,int,int]:`

Este algoritmo al igual que el anterior, busca hallar el máximo común divisor entre dos números enteros. Sin embargo, aquí, usaremos el Teorema de Bézout y algoritmo de Euclides extendido, teorema que afirma que el máximo común divisor entre dos números puede expresarse como una combinación lineal de esos números. Este teorema nos dice que si desarrollamos la ecuación $mx + ny = d = (m,n)$ encontramos unas coordenadas x,y que son solución de la ecuación anterior.

La función recibe los argumentos n y m números enteros y devuelve una tupla de enteros; d , como mcd entre n y m , x , como coordenada de m e y , como coordenada de n .

El desarrollo del algoritmo se da de la siguiente manera; primeramente, se comprueba el caso de que m o n sean 0, ya que en caso de que alguno de los dos lo sea, sabemos que entonces la coordenada buscada del número que sea distinto de 0 es 1 y que, por tanto, d , que equivale al máximo común divisor entre n y m , sería el número distinto de 0, ya que el mcd divisor entre 0 y un número es el número. Esto se puede comprobar también con el algoritmo de Euclides ya que sabemos que si aplicamos el algoritmo para calcular el máximo divisor entre (u,v) siendo u,v números enteros, usando $r = b - ka$ siendo k un número entero, entonces el máximo de los dos números disminuye en cada paso hasta obtener o bien $(0,p)$ siendo p un número entero distinto de 1 y siendo a su vez el máximo común divisor entre (u,v) . Llegamos así al caso que planteábamos anteriormente, donde si n o m son 0 queda probado lo que comentábamos. En caso de que no lo sean vamos reduciendo el número mayor de manera recursiva y estableciendo a su vez las coordenadas asociadas a cada número aplicando el método explicado. Finalmente, devolvemos el máximo común divisor entre n y m y sus respectivas coordenadas. La implementación de este algoritmo es recursiva al tratarse de ejecutar el mismo proceso hasta que llegamos a la condición de parada (n o m iguales a 0).

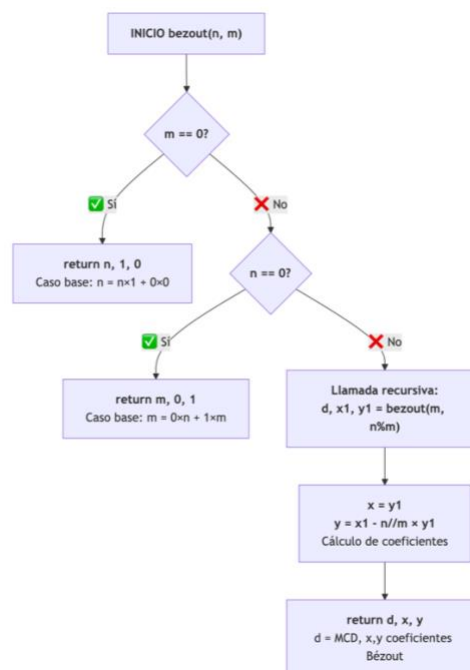


Figura 6. Diagrama de flujo de la función bezout ($n:\text{int}, m:\text{int}$) $\rightarrow$ tuple[int, int, int]. Elaboración propia.

El diagrama de flujo deja constancia de las condiciones iniciales si m o n sean 0 o no, en caso de que lo sean retornamos lo que corresponda, si no se da ninguno de los dos casos, aplicamos bezout para

sacar las coordenadas x,y de manera recursiva y finalmente devolvemos las coordenadas y el mcd de a y b.

- `mcd_n(n:List[int]) -> int`:

El algoritmo implementado se basa en la propiedad asociativa del máximo común divisor, que establece que el máximo común divisor de tres o más números es equivalente a hallar recursivamente el máximo común divisor del máximo común divisor de los primeros pares con el resto de los números (es decir, $\text{mcd}(a, b, c) = \text{mcd}(\text{mcd}(a, b), c)$). Esta propiedad nos permite aplicar de manera sucesiva el algoritmo de Euclides, enunciado previamente, por pares.

La implementación comienza verificando si la lista está vacía, caso en el que retorna 0 (por convención). Luego inicializa el resultado con el primer elemento de la lista y empieza a iterar sobre los elementos restante, a medida que va actualizando el valor de MCD con cada iteración.

Una optimización clave con respecto a la función `mcd(a, b)` es el corte temprano cuando llega al valor 1, lo que mejora la eficiencia para listas largas.

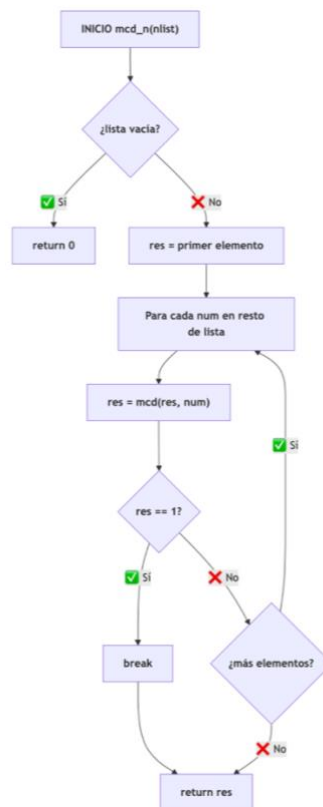


Figura 7. Diagrama de flujo de la función `mcd_n(n:List[int]) -> int`. Elaboración propia.

Como podemos ver en el diagrama de flujo, hay un flujo lineal que empieza con la verificación de la lista y la inicialización del resultado, procediendo entonces a la iteración de los elementos restantes. La conexión directa entre las llamadas `mcd()` sucesivas nos permite ver la aplicación de la propiedad asociativa, fundamental para el funcionamiento de la función.

- `bezout_n(nlist:List[int]) -> Tuple[int, List[int]]`:

El algoritmo generaliza la identidad de Bézout para poder aplicarlo sobre múltiples números, basándose en la propiedad que establece que “el máximo común divisor de una lista de enteros puede expresarse como la menor combinación lineal de dichos números” (Palacios, 2024).

La implementación sigue un enfoque iterativo que empieza con una verificación de si la lista que ha recibido está vacía, retornando la tupla (0, []) según convención. Para cada número adicional en la lista, calcula una versión ampliada de la función bezout().

La clave del algoritmo reside en la actualización de los coeficientes: multiplica todos los coeficientes y los añade al sistema, garantizando que siguen siendo combinación lineal. Esto garantiza que al final se cumpla la ecuación $d = a_1x_1 + \dots + a_nx_n$ para toda la lista.

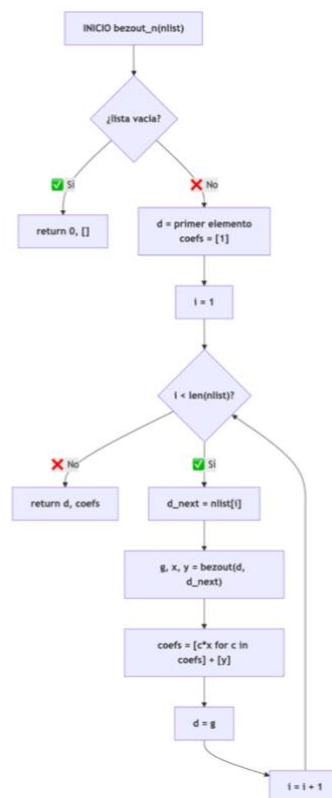


Figura 8. Diagrama de flujo de la función `bezout_n(nlist:List[int]) -> Tuple[int, List[int]]`. Elaboración propia.

El diagrama muestra el flujo iterativo, que se inicializa estableciendo el primer elemento como base para la posterior combinación lineal. En cada iteración, vemos como se sigue el proceso de calcular la identidad de Bézout entre el máximo común divisor acumulado y el siguiente número de la lista, a la vez que se actualizan los valores de los coeficientes en cada paso para mantener la validez de la ecuación. El bucle implementado garantiza que todos los números de la lista se implementen en la combinación lineal final.

- `coprimos(n:int, m:int) -> bool`:

El algoritmo de esta función se basa en la definición fundamental de números coprimos: dos números enteros n y m son coprimos (también llamados primos relativos) “si y solo si su máximo

común divisor es exactamente 1” (Nápoli, 2020). Esta definición, ya enunciada en los trabajos fundamentales de Euclides, es la base teórica de este algoritmo.

El algoritmo que rige esta función es tan elegante como simple: en lugar de implementar una verificación directa mediante factorización, el algoritmo aprovecha la función $\text{mcd}(n, m)$, previamente implementada, reutilizando código e implementando principios de abstracción de código. Esto no solo reduce la redundancia de líneas, sino que fuerza la consistencia: todas las operaciones relacionadas con divisibilidad utilizan el mismo algoritmo subyacente.

Además, presenta un excelente manejo de casos especiales: al ser el número 1 coprimo con cualquier otro número, cuando la función reconoce que alguno de los factores es 1, devuelve inmediatamente True, ya que el uno es coprimo con cualquier otro número.

Hereda la complejidad algorítmica del algoritmo de Euclides, así como su eficiencia. Esto significa que, para números extremadamente grandes, el tiempo de ejecución es mínimo.

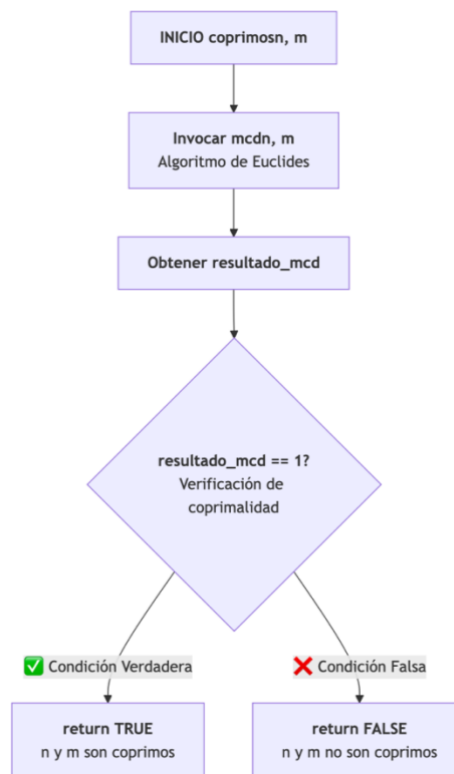


Figura 9. Diagrama de flujo para la función $\text{coprimos}(n:\text{int}, m:\text{int}) \rightarrow \text{bool}$. Elaboración propia.

- $\text{potencia_mod_p}(\text{base}:\text{int}, \text{exp}:\text{int}, \text{p}:\text{int}) \rightarrow \text{int}$:

Este algoritmo se basa en la técnica de exponenciación por cuadrados repetidos (conocido como método de “exponenciación binaria” o algoritmo de “right-to-left binary exponentiation”). Esta técnica tiene sus raíces en libros de matemáticas antiguas, sin estar clara su autoría concreta, pero fue formalizada por Knuth en su libro “The Art of Computer Programming”.

El algoritmo inicia con una validación de los casos especiales, concretamente en caso 0^0 y los exponentes negativos. Para este último caso, aplicamos una conversión mediante el cálculo de la inversa modular, usando así de manera recursiva el mismo algoritmo que para exponentes positivos.

Usamos la representación de tipo binario del exponente para descomponer el cálculo en cuadrados y multiplicaciones.

Además, prevemos el desbordamiento aritmético mediante la reducción modular continua mediante el cálculo del resto de la división por el módulo.

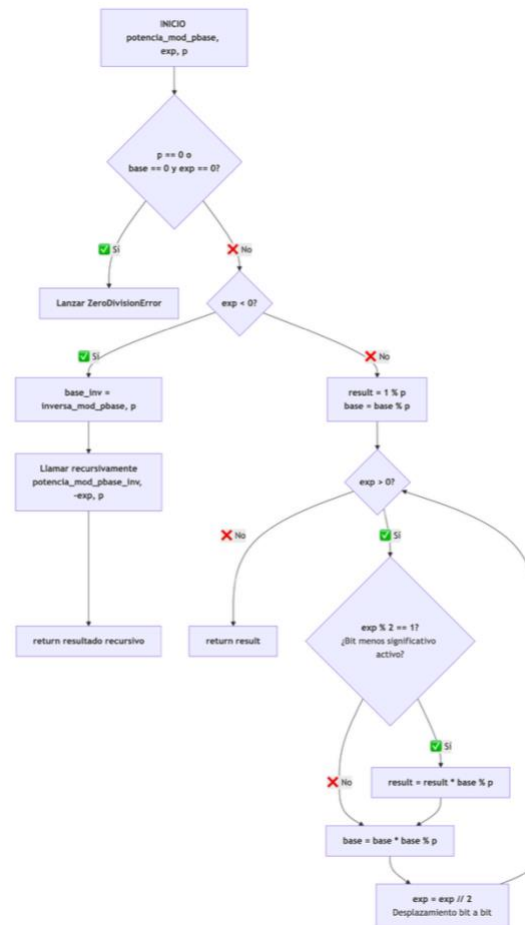


Figura 10. Diagrama de flujo para la función potencia_mod_p(base:int, exp:int, p:int) -> int. Elaboración propia.

El diagrama de flujo deja constancia del bucle principal, donde vemos el escaneo bit a bit del exponente. Observamos que la condición de terminación $exp > 0$ garantiza que el proceso continua hasta que confirma que todos los bits han sido procesados. También vemos como la reducción continua del módulo mantiene la estabilidad del proceso y previene *overflows* del sistema cuando se introducen números grandes.

- inversa_mod_p(n:int, p:int) -> int

Volvemos a utilizar el algoritmo extendido de Euclides iterativos, con una cierta base en el teorema de Bézout para determinar cuándo dos números son coprimos. El **núcleo iterativo** del algoritmo realiza sucesivas divisiones euclidianas, actualizando en cada paso tanto los valores de a y b como los

coeficientes x_0 y x_1 . La clave matemática reside en que en cada iteración se mantiene la invariante $x_0 \cdot n + x_1 \cdot p = a$, lo que garantiza que al final del proceso, cuando a sea el MCD(n, p), x_0 será el coeficiente de Bézout correspondiente a n .

La implementación comienza validando el módulo de p , rechazando cuando el módulo tiene el valor 0 (carece de sentido matemático cuando realizamos cálculos de aritmética modular). Inicializa entonces los valores para aplicar los teoremas. Además, comprueba si el mcd de los valores no es 1, que entonces lanza una excepción indicando que no es invertible.

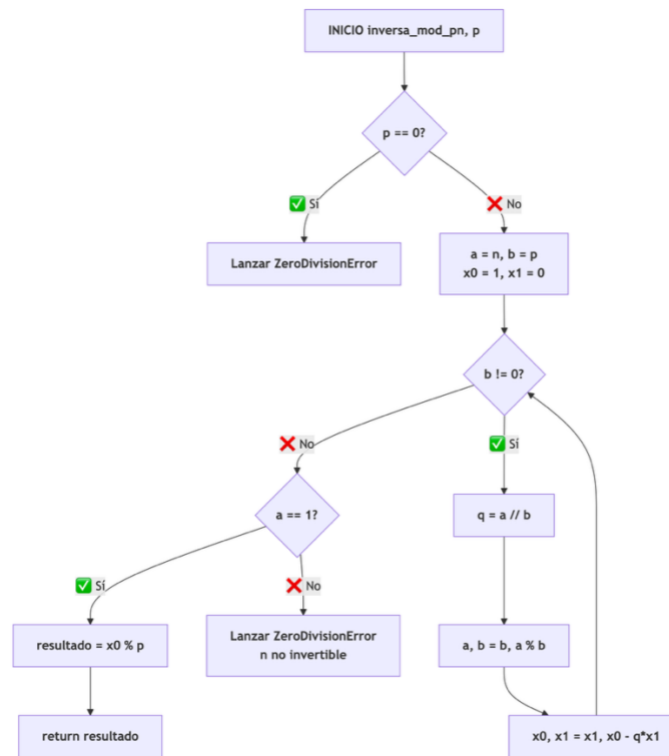


Figura 11. Diagrama de flujo de la función `inversa_mod_p(n:int, p:int) -> int`. Elaboración propia.

Observamos aquí la estructura en tres etapas del algoritmo: inicialización, core euclidiano y la verificación del resultado. En el bucle principal se calcula el cociente q de la división a/b , luego se actualizan a y b mediante el algoritmo estándar de Euclides, y simultáneamente se actualizan los coeficientes x_0 y x_1 usando la relación de recurrencia de Bézout. Esta actualización simultánea es crucial para mantener la invariante algebraica durante la ejecución del proceso. La condición de terminación ($b = 0$) marca el final del algoritmo de Euclides, momento en el cual a contiene el MCD(n, p). La verificación posterior ($a = 1$) es esencial para garantizar la existencia de la inversa modular. Si esta condición no se cumple, el algoritmo rechaza correctamente el cálculo, indicando la no invertibilidad de n módulo p .

- `euler(n:int) -> int`:

Este algoritmo se basa en la formula productoria de Euler : $\phi(n) = n \times \prod (1 - 1/p)$ para todos los factores primos p de n (Prego, 2009). Esta fórmula, descubierta por Leonhard Euler en el siglo XVIII permite calcular $\phi(n)$ sin necesitar verificar la coprimalidad de los números menores que n .

Comienza, como en el resto de las funciones, con un manejo de casos especiales donde retorna 0 para enteros menores que 1. Luego inicializa el valor de n , que ira reduciéndose conforme vamos identificando primos.

La optimización clave reside en el tratamiento inicial del factor q , esto nos permite avanzar en incrementos de 2 reduciendo así a la mitad de las verificaciones necesarias. Luego, el algoritmo itera sobre los posibles factores primos impares restantes desde 3 hasta $\sqrt{n}$. Para cada factor primo p encontrado, aplica la transformación $\phi(n) = \phi(n) \times (1 - 1/p)$, manteniendo la aritmética entera.

El caso final maneja la situación donde, después de verificar todos los factores hasta $\sqrt{n}$, queda un número $n > 1$, que necesariamente debe ser primo. Esta observación matemática permite terminar el algoritmo sin necesidad de verificar divisores más allá de $\sqrt{n}$, optimizando significativamente el cálculo para números primos grandes.

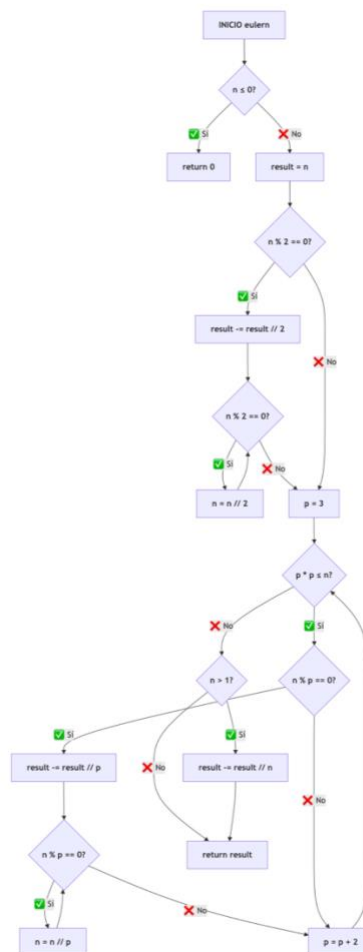


Figura 12. Diagrama de flujo de la función $euler(n:int) \rightarrow int$. Elaboración propia.

Observamos la estructura secuencial y optimizada del algoritmo. El bucle principal para primos impares demuestra la eficiencia del algoritmo: al verificar solo hasta $\sqrt{n}$ y avanzar en pasos de 2, reduce drásticamente el número de iteraciones necesarias. La estructura anidada de bucles para cada factor primo garantiza que se eliminen todas las ocurrencias de cada primo antes de proceder al siguiente. La condición de terminación del bucle principal ($p^2 \leq n$) implementa el corte óptimo basado en la propiedad que, si n tiene un factor primo, debe tener uno menor o igual que $\sqrt{n}$. La etapa final maneja el caso del posible primo residual de manera eficiente, aplicando la fórmula de Euler una última vez si es necesario.

- `legendre(n:int, p:int) -> int:`

La implementación de esta función se basa en el criterio de Euler establece que $(n/p) \equiv n^{((p-1)/2)} \pmod{p}$, aplicándolo mediante el método de exponenciación modular.

La optimización clave del algoritmo reside en el manejo de casos especiales antes de realizar la exponenciación costosa. Si $n \equiv 0 \pmod{p}$, retorna inmediatamente 0. Si $n \equiv 1 \pmod{p}$, retorna 1 sin necesidad de cálculo adicional. Para $n \equiv -1 \pmod{p}$, aplica la fórmula específica para $(-1/p)$ que depende de $p \pmod{4}$, usando la operación bit a bit $p \& 3$, equivalente a $p \% 4$ pero más eficiente.

Solo cuando ninguno de estos casos especiales aplica, el algoritmo procede a calcular la exponenciación modular usando la función `potencia_mod_p()`. El exponente utilizado es $(p-1)/2$, que se calcula mediante desplazamiento bit a bit ($>> 1$) por eficiencia.

- Este algoritmo se basa en resolver un sistema de congruencias dadas 3 listas de números enteros, sabiendo que los módulos son coprimos dos a dos. Cada a_i (variable) está asociada a un b_i (término independiente) con módulo p_i , yendo así por posiciones en la lista, por lo que las listas han de ser de la misma longitud. Por ello, lo primero que haremos será ver cuál es la longitud de las listas, si las listas no son de la misma longitud, el programa lanza un error: `IncompatibleEquationError` con el mensaje “las tres listas deben tener la misma longitud”. Lo mismo sucede si la longitud de alguna de las listas

es 0. De nuevo el programa lanzará un error: `IncompatibleEquationError` con el mensaje “se debe proporcionar al menos una ecuación”.

Lo siguiente que deberemos hacer será construir nuestra ecuación de congruencias y para ello seleccionamos a, b y p asociados a la posición en la que estemos trabajando y lanzaremos un error si encontramos que p es un número negativo, ya que los módulos siempre son positivos. Para simplificar la ecuación calcularemos el máximo común divisor entre a y p y procederemos a dividir los 3 componentes de la ecuación (a, b, p) entre ese máximo común divisor. Si b no fuera divisible por el máximo común divisor, entonces la ecuación no tiene solución, de nuevo lanzaríamos un error. En caso contrario, dividimos a, b y p por el máximo común divisor de a y p .

Para dejar la x sola, multiplicaremos por la inversa de a a b , tal que $x = a^{-1} * b \bmod(p)$. Haciendo uso de la función `inversa_mod_p` calcularemos la inversa de a respecto del módulo p , si p es 0 entonces lanza el error `ZeroDivisionError` que captura el error que se formaría al dividir un número (a) entre 0. Si no salta el error, entonces calculamos la solución de la ecuación ($x = a^{-1} * b \bmod(p)$). Esa solución la añadimos a la lista de soluciones junto con el módulo p (simplificado) asociado a la ecuación. Si solo hay una ecuación, la solución es directa y la devolvemos junto con su módulo p asociado.

Resolvemos el sistema haciendo uso del Teorema Chino del Resto, establece que dado un sistema de congruencias lineales donde los módulos son coprimos entre sí (es decir, su máximo común divisor es 1), existe una solución única para la variable entera en un rango determinado. Seleccionaremos la solución de la primera ecuación y su módulo y la siguiente solución y módulo mediante un `for`. Comprobaremos por si acaso que ambos módulos son coprimos usando la función `coprimos`. En caso de que no lo fueran, lanzamos un mensaje donde decimos que este supuesto no es válido pues viola la condición inicial y por tanto para dar una solución a esa ecuación llamaríamos a la siguiente función, donde se resuelven sistemas de congruencias para módulos coprimos o no coprimos. En caso de que sí fueran coprimos, resolvemos el sistema aplicando el Teorema Chino con las dos ecuaciones y hallamos los coeficientes mediante la función `bezout()` para la ecuación formada por los módulos que estamos analizando. Si el máximo común divisor entre ambos módulos no es 1, lanzamos un error devolviendo el mensaje de que los módulos no son coprimos. Sacamos la solución ($x = a_1 * x_1 * N_1 + a_2 * x_2 * N_2$) de la ecuación haciendo uso de la ecuación actual que estamos estudiando y la siguiente y los coeficientes obtenidos por `bezout()`. Siguiendo el Teorema Chino del resto, calculamos el nuevo módulo como producto de los dos que estamos analizando y la solución final, como solución (módulo_nuevo) y actualizamos el módulo al nuevo que hemos obtenido aplicando el Teorema. Finalmente devolvemos una solución (resto) que es mayor que cero y menor que el módulo nuevo. Si el resto fuera negativo, le sumamos el módulo nuevo para convertirlo en positivo. Llevamos este proceso a cabo con todas las demás ecuaciones del sistema.

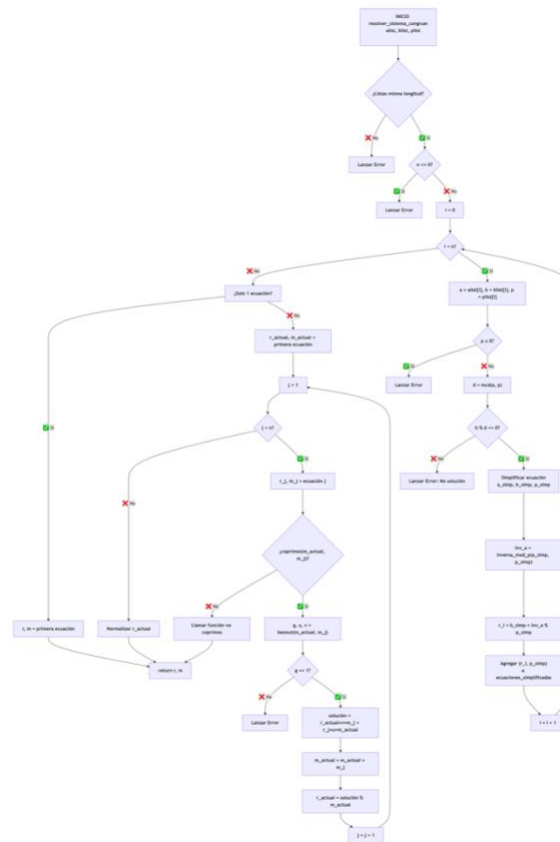


Figura 14. Diagrama de flujo de la función resolver_sistema_congruencias(alist: List[int], blist: List[int], plist: List[int]) -> tuple[int, int] Elaboración propia.

El diagrama de flujo deja constancia de las verificaciones que llevamos a cabo al comienzo, donde si se ve si las longitudes de las listas son o no iguales y si el tamaño de las listas es al menos 1. Si se dan estas condiciones comenzamos a ejecutar el programa, analizando el valor de p y dando resultado en función de eso. Procedemos a ver si la ecuación puede tener solución mediante la simplificación de los coeficientes. Buscamos la inversa de a , devolviendo un error si no existe o aplicando Bézout en caso contrario. Verificamos que los módulos son coprimos y se denota de forma clara como se devuelve un error en caso de que no lo sean. Finalmente sacamos la solución particular y actualizamos los módulos y la solución inicial. Ajustamos la solución y la devolvemos junto con su módulo.

- resolver_sistema_congruencias_no_seguro_coprimos(*ecuaciones: List[int]) -> tuple[int, int]:

El procedimiento seguido en este algoritmo es muy similar al algoritmo anterior, solo que ahora no podemos asumir que los módulos sean coprimos. Al igual que en el caso anterior devolvemos la solución y el módulo asociado. De nuevo comenzaremos viendo la longitud de la lista que nos han proporcionado. Si estuviera vacía lanzamos un mensaje donde especificamos que es necesario proporcionar al menos una ecuación. Lo primero que haremos será simplificar cada ecuación de manera de individual, siempre y cuando la longitud de esa ecuación sea tres, ya que, si no, devolvemos

un mensaje afirmando que se deben proporcionar tres elementos. Separamos cada elemento de la ecuación (a = variable, b = término independiente, p = módulo) y de nuevo veremos si p es positivo o negativo. En caso de que sea negativo, lanzamos un mensaje diciendo que el módulo siempre ha de ser positivo.

Estudiaremos primero un caso especial, donde $a = 0$, es decir, $0x$ si el módulo de $b = 0$ entonces cualquier x es solución, por lo que el resultado es válido y por tanto lo añadimos a la lista de soluciones. Si no fuera así, lanzamos un mensaje de error al no tener solución la ecuación. El proceso siguiente es igual que en el algoritmo anterior; simplificamos la ecuación, viendo si b es divisible por el mcd de a y p , buscando la inversa de a si fuera así y devolviendo sino un mensaje de error ya que en ese caso la ecuación no tiene solución. De nuevo vemos si existe la inversa de a , capturando el error `ZeroDivisionError` si el módulo es 0 o la base y el exponente son 0. Posteriormente, encontramos la solución particular a esa ecuación, si solo hay una ecuación, la solución es directa y devolvemos el resto de la división del factor r entre m y el módulo.

Si hay más de una ecuación, vamos a ir combinando las ecuaciones de manera iterativa comenzando con la primera ecuación y mediante el `for` iremos analizando las siguientes. En la primera iteración analizamos la primera y segunda ecuación y veremos si sus módulos son o no coprimos, llamando a la función `mcd` entre los dos módulos (primera y segunda ecuación) y si ese `mcd` es 1 `son_coprimos` será `True`, en caso contrario (módulo de la primera y segunda ecuación no son coprimos), valdrá `False`. Si son coprimos, seguiremos el razonamiento llevado a cabo en la función anterior, es decir, utilizamos el Teorema Chino del resto para hallar la solución de la ecuación y el módulo nuevo que será producto de los dos módulos que estamos estudiando, actualizamos la solución y el módulo y volveríamos a iterar. En caso de que no sean coprimos, verificaremos primero si el sistema es compatible o no calculando el `mcd` de los módulos y si las soluciones no son divisibles por d dando el mismo resto, el sistema no es compatible, lanzamos un mensaje de error diciendo que el sistema no es compatible al no ser las ecuaciones compatibles. Aplicamos Bézout para sacar el `mcd` entre los módulos y las coordenadas de los módulos y calculamos la diferencia entre las soluciones que estamos estudiando, la cual debería ser múltiplo del `mcd` de los módulos (resto 0) para que el sistema sea compatible, si no, lanzamos un mensaje por pantalla por la incompatibilidad del sistema. Si ese resto es cero, entonces calculamos cuál es el factor que hace que al dividir la diferencia entre el `mcd` de los módulos el resto sea 0. Sacamos a su vez la solución particular de este sistema.

Calculamos el módulo nuevo como mínimo común múltiplo entre ambos módulos (esto lo hacemos así porque los módulos son coprimos), la solución actual como solución (módulo módulo_nuevo) y actualizamos el módulo actual (que es el que se correspondía inicialmente como el módulo de la primera ecuación). Aseguraremos tanto aquí como al final que la solución se encuentre en el rango correcto, es decir, que sea número positivo y menor que el módulo. Finalmente verificamos que la solución satisface la ecuación original recorriendo con un `for` la lista de ecuaciones dada, separando a, b, p y trabajando con `encontrado` (booleano que empieza en `False`). Si al multiplicar a por la solución y calcular su módulo respecto p no es igual que hacer el módulo de b respecto de p , entonces no hemos encontrado la solución y tendremos que volverla a buscar recorriendo con un `for` los números desde el 0 hasta el mínimo entre p y el módulo nuevo que habíamos sacado anteriormente. Obtenemos un candidato sumándole a la solución actual un múltiplo del módulo. De nuevo haríamos la comprobación anterior, si al multiplicar a por el candidato y hacer el módulo respecto de b sale el mismo resultado que hacer el módulo de b respecto de p , si así fuera, la solución pasa a ser el candidato y actualizamos el módulo al mínimo común múltiplo del módulo actual y p , encontrado pasa

a ser True y aseguramos que la solución final esté normalizada. Si el candidato no fuera la solución buscada, seguimos iterando o bien hasta que encontremos la solución o bien hasta que finalmente salgamos del for. En caso de no encontrar ninguna solución lanzamos un mensaje por pantalla afirmando que la solución encontrada no satisface la ecuación y que por ello no es válida. Repetiríamos este proceso con todas las ecuaciones de la lista de ecuaciones dada.

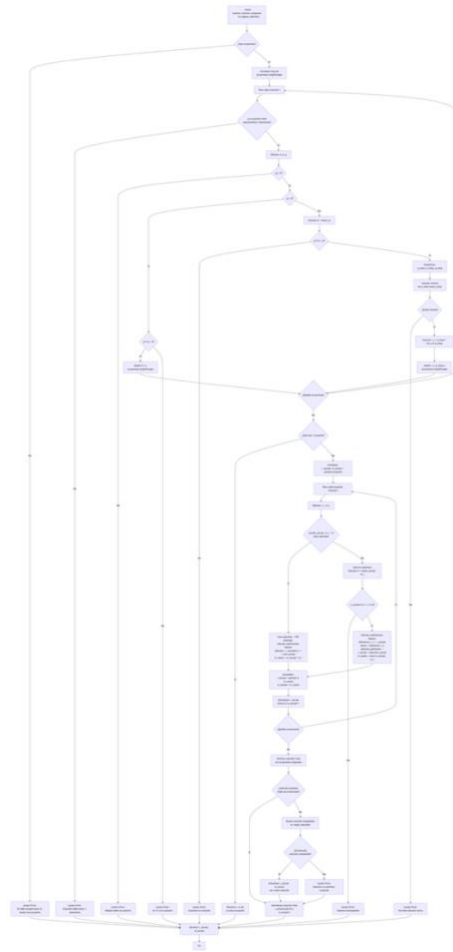


Figura 15. Diagrama de flujo de la función `resolver_sistema_congruencias_no_seguro_coprimos(*ecuaciones: List[int]) -> tuple[int, int]` Elaboración propia

El diagrama de flujo deja constancia del análisis de las condiciones iniciales; que la longitud de las ecuaciones sea mayor que 0, devolviendo un error si no es así. Pasamos a simplificar cada ecuación individualmente. Verificamos primero que el módulo sea positivo, si no, lanzamos un error. Pasamos a analizar el caso especial $a = 0$. Si el módulo de b respecto de p es 0, añadimos la solución a la lista, si no, lanzamos un error. Si a no es 0, simplificamos cada ecuación siguiendo el proceso de la función 11. Si no se lanza el error `ZeroDivisionError` o `IncompatibleEquationError`, obtenemos la solución particular y la añadimos a la lista. Si la longitud de la lista es 1, devolvemos directamente el resultado, si no, estudiamos usando un for las ecuaciones. Verificamos si los módulos que estamos estudiando son coprimos, si lo son, seguimos el proceso explicado en la función 11. Si no, buscamos el máximo común divisor entre los módulos y vemos la compatibilidad. Si no son compatibles, lanzamos error, si lo son, resolvemos el sistema usando la función bezout. Calcularemos la diferencia entre módulos y

verificaremos la divisibilidad, lanzando error en caso de que no sean divisibles. Si lo son, hallamos la solución particular usando mcd, mcm y la solución inicial de la que partíamos. Verificamos que la solución se encuentra en el intervalo $[0, p-1]$ y pasamos a la verificación final de la solución. De manera iterativa (for) iremos recorriendo las ecuaciones separando los 3 elementos (a,b,p) y miramos si la solución actual satisface la ecuación original. Si es así, la normalizamos y la devolvemos junto con su módulo. Si no es así seguimos buscando la solución en otro for que va desde 1 hasta el mínimo entre el módulo actual y p. Si hay más elementos seguimos buscando hasta que encontremos solución o hasta que ya no haya más elementos en cuyo caso devolvemos un error.

- raiz_mod_p(n:int,p:int)->int:

Este algoritmo tiene como funcionalidad calcular raíces n del módulo p siendo n un entero y p un número primo. Devolvemos un entero entre 0 y $(p-1)$ tal que $x^2 = n \pmod{p}$ tal que sea raíz.

Comenzamos calculando el módulo de n respecto de p y analizando los casos triviales, si n es 0 entonces la solución es 0, y si p es 2, entonces la solución es n. Usamos la función legendre() que nos ayudará a saber si n tiene raíz cuadrada módulo p, si al llamar esta función el resultado es distinto de 1, entonces no existe solución, ya que tal y como se ha explicado en la función legendre() si vale 1, n es un cuadrado módulo p.

Si la función legendre() ha devuelto un 1, probamos un caso especial donde $p = 3 \pmod{4}$. Llamamos a la función potencia_mod_p() (explicada más arriba), con base n, exponente la división exacta de $p+1$ entre cuatro y como módulo la p. Si al elevar al cuadrado la x devuelta por la función potencia_mod_p() y calcular su módulo respecto de p da n, devolvemos la x, si no, no. Si no hemos devuelto x probamos una búsqueda lineal recorriendo con un for los números de 1 a $p-1$ y viendo si al calcular su cuadrado y hacer el módulo respecto de p da n, en ese caso, la x es válida y la devolvemos. Si no, seguimos iterando hasta que encontremos solución o lleguemos hasta $x = p-1$. En caso de no haber encontrado solución en ningún caso lanzamos un mensaje por pantalla afirmando que no se ha encontrado solución para n módulo p, aunque este caso, no se debería dar.

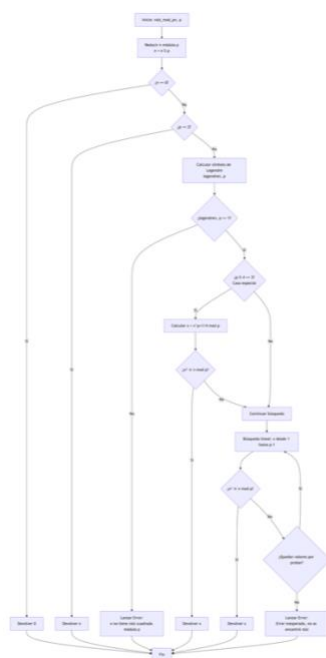


Figura 16. Diagrama de flujo de la funcion raiz_mod_p(n:int,p:int)->int Elaboracion propia

El diagrama de flujo deja constancia de los primeros análisis de los casos especiales, devolviendo resultados en caso de que estemos en un caso especial y pasando a analizar con detenimiento si no fuera así. Verificamos que existe solución con la función legendre y analizamos un caso especial ($p \% 4 = 3$). Si estamos en ese caso especial, lo resolvemos aplicando la función potencia_mod() y devuelvo resultado si la solución es válida. Si no estamos en el caso especial, recorreremos con un for las soluciones posibles sacando el candidato cuadrado, vemos si cuadra. Si cuadra devolvemos el resultado, si no, seguimos iterando hasta encontrar solución o salir del bucle.

- ecuacion_cuadratica(a:int,b:int,c:int,p:int)->tuple[int,int]:

El objetivo del algoritmo es hallar si es posible las dos soluciones repetidas o no a la ecuación cuadrática módulo p. La función recibe tres números enteros que serán los coeficientes de la ecuación cuadrática ($ax^2 + bx + c$) y p, número entero que hace referencia al módulo y devuelve una tupla con las dos soluciones a la ecuación estando acotadas ambas entre 0 y p-1.

Comenzaremos reduciendo los coeficientes módulo p y analizamos el caso especial donde si a,b,c son 0 entonces la solución que devolvemos es (0,0), si a y b son 0 pero c no, entonces lanzamos un mensaje de error afirmando que la ecuación es incompatible. Si solo a es 0, resolvemos la ecuación como si fuera una ecuación lineal, haciendo uso de la función inversa_mod_p usando b (coeficiente asociado a la x) y multiplicamos $-c \cdot \text{inversa}$ de b y calculamos su módulo respecto p hallando así la solución de la ecuación y devolviendo una tupla (x,x) siguiendo el formato dado, con la x sacada realizada esta operación.

Calcularemos el discriminante ($b^2 - 4ac$) como b^2 es igual al cuadrado módulo respecto de p y $ac4$ es cuatro por a y por c módulo respecto de p, y finalmente el discriminante es el módulo de la diferencia de $b^2 - ac4$ respecto de p. Si el discriminante es negativo le sumamos p, si es cero, implica que habrá raíz doble, ya que el discriminante es lo que va dentro de la raíz cuando aplicamos la fórmula de la

ecuación de segundo grado por lo que usando la función inversa_mod_p utilizando 2 por a módulo de p multiplicando ese resultado por $-b$ y haciendo el módulo respecto de p llegaremos a la x pedida, si esa x es negativa, le sumamos p para que sea positiva y devolvemos la tupla (x,x) . Si el discriminante no fuera 0, probamos a calcular su raíz aplicando la función anterior. Si no existe solución, lanzamos un mensaje, si por el contrario existe, calculamos la inversa de a y aplicando la fórmula cuadrática calculamos $x_1 = ((-b + \text{raíz_d}) * \text{inv_2a}) \% p$ y $x_2 = ((-b - \text{raíz_d}) * \text{inv_2a}) \% p$, al multiplicar por la inversa de a es como si estuviéramos dividiendo por $2a$, siguiendo la fórmula cuadrática. Finalmente, nos aseguramos de que las soluciones son positivas y menores que p y devolvemos la tupla (x_1, x_2) .

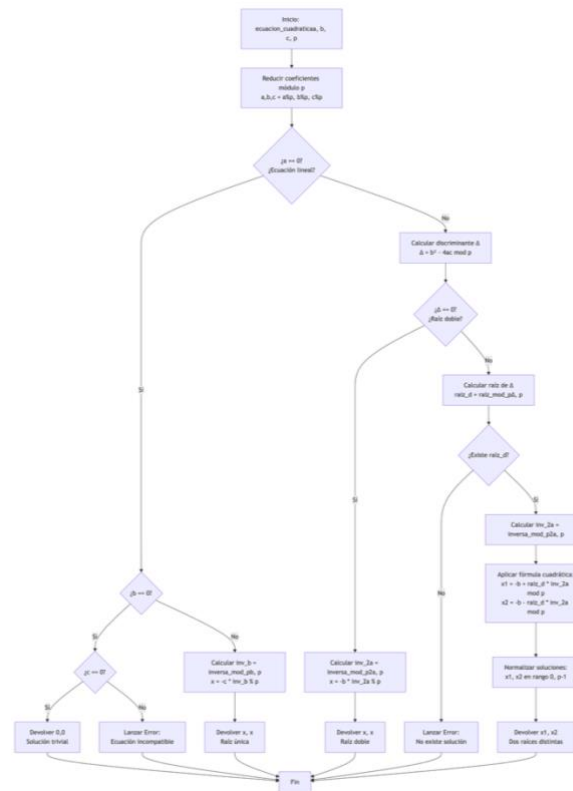


Figura 17. Diagrama de flujo de la función `ecuacion_cuadratica(a:int,b:int,c:int,p:int)->tupla[int,int]` Elaboración propia.

El diagrama de flujo deja constancia de la reducción inicial de los coeficientes de la ecuación respecto módulo p, analizamos el caso especial donde a sea 0, en caso de que estemos en el caso, vemos si b es 0, si es así, vemos si c es 0, si c es así, devolvemos la solución $(0,0)$, si c no es 0, devolvemos un error y si b no es 0, sacamos la solución haciendo uso de la inversa y la fórmula cuadrática devolviendo (x,x) . Si a no es 0 analizamos el discriminante; si el discriminante es 0, la solución es doble y la sacamos utilizando el caso $b = 0$. Si el discriminante fuera negativo le sumamos p y pasamos a analizar si tiene o no raíz. En caso de que no, lanzamos un error, en caso de que sí, terminamos de resolver la ecuación aplicando paso a paso la fórmula cuadrática devolviendo (x_1, x_2) .

MANUAL DE USUARIO

COMANDOS DE USO

El sistema IMATLAB ofrece 14 comandos principales organizados en categorías:

1. TEORÍA DE NÚMEROS BÁSICA

- `primo(n)` - Verifica si un número es primo
- `primos(a, b)` - Lista primos en el intervalo $[a, b]$
- `factorizar(n)` - Descompone en factores primos
- `coprimos(n, m)` - Verifica si dos números son coprimos

2. ALGORITMOS EUCLIDIANOS

- `mcd(a, b)` - Máximo común divisor de dos números
- `mcd(n1, n2, ..., nk)` - MCD de múltiples números
- `bezout(a, b)` - Identidad de Bézout para dos números
- `bezoutn(n1, n2, ..., nk)` - Bézout extendido para múltiples números

3. ARITMÉTICA MODULAR

- `pow(base, exp, p)` - Exponenciación modular
- `inv(n, p)` - Inversa modular
- `euler(n)` - Función ϕ de Euler
- `legendre(n, p)` - Símbolo de Legendre

4. ECUACIONES Y SISTEMAS

- `raiz(n, p)` - Raíz cuadrada modular
- `ecCuadratica(a, b, c, p)` - Ecuación cuadrática modular
- `resolverSistema([a1,b1,p1], [a2,b2,p2], ...)` - Sistema de congruencias

DIFERENTES MODOS DE USO

El sistema IMAT-LAB cuenta con dos modos distintos de uso, dependiendo de cuál sea el uso que se le pretenda dar:

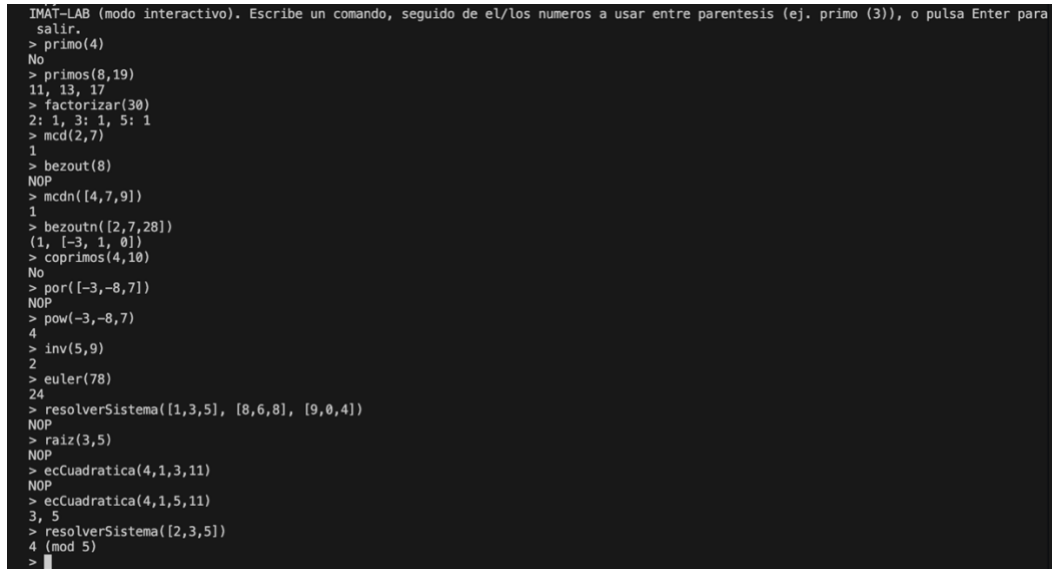
- MODO INTERACTIVO

El modo interactivo está diseñado para usuarios que desean explorar las funcionalidades del sistema de manera inmediata y pedagógica. Al ejecutar el programa sin argumentos, se inicia una sesión interactiva donde el usuario puede introducir comandos uno por uno y ver los resultados instantáneamente.

Las características principales de este modo son:

- Prompt personalizado: Aparece el símbolo `>` indicando que el sistema está listo para recibir comandos,
- Feedback inmediato: Cada comando se ejecuta y muestra su resultado en la misma línea,
- Sesión continua: Permite realizar múltiples operaciones sin reiniciar el programa,

- Manejo de errores en tiempo real: Los errores se muestran inmediatamente con mensajes comprensibles, y
- Salida simplificada: Ideal para aprendizaje y experimentación matemática.



```

IMAT-LAB (modo interactivo). Escribe un comando, seguido de el/los numeros a usar entre parentesis (ej. primo (3)), o pulsa Enter para
salir.
> primo(4)
No
> primos(8,19)
11, 13, 17
> factorizar(30)
2: 1, 3: 1, 5: 1
> mcd(2,7)
1
> bezout(8)
NOP
> mcdn([4,7,9])
1
> bezoutn([2,7,28])
(1, [-3, 1, 0])
> coprims(4,10)
No
> par([-3,-8,7])
NOP
> pow(-3,-8,7)
4
> inv(5,9)
2
> euler(78)
24
> resolverSistema([1,3,5], [8,6,8], [9,0,4])
NOP
> raiz(3,5)
NOP
> ecCuadratica(4,1,3,11)
NOP
> ecCuadratica(4,1,5,11)
3, 5
> resolverSistema([2,3,5])
4 (mod 5)
>

```

Figura 18. Ejemplo de ejecución IMAT-LAB en modo interactivo. Elaboración propia.

- MODO BATCH (POR LOTES)

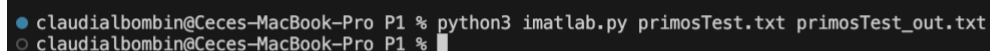
El modo batch está optimizado para procesamiento automatizado y de grandes volúmenes de operaciones. Se activa cuando se proporcionan dos argumentos: un archivo de entrada y un archivo de salida.

Características principales:

- Procesamiento masivo: Puede ejecutar miles de comandos en una sola invocación,
- Automatización: Ideal para scripts, pruebas automatizadas y procesamiento por lotes,
- Eficiencia: Minimiza el overhead de inicialización al procesar múltiples comandos,
- Registro de resultados: Todos los outputs se guardan en un archivo para análisis posterior, y
- Manejo robusto de errores: Continúa procesando incluso si algunos comandos fallan.

Flujo de procesamiento:

1. Lee línea por línea del archivo de entrada
2. Ejecuta cada comando secuencialmente
3. Escribe los resultados en el archivo de salida
4. Maneja errores consistentemente (NE para errores matemáticos, NOP para comandos inválidos)



```

● claudialbombin@Ceces-MacBook-Pro P1 % python3 imatlab.py primosTest.txt primosTest_out.txt
○ claudialbombin@Ceces-MacBook-Pro P1 %

```

Figura 19. Ejemplo de ejecución IMAT-LAB en modo batch (por lotes). Elaboración propia.

CONCLUSIONES

El desarrollo de IMAT-LAB ha culminado exitosamente, superando los objetivos iniciales y estableciendo un sólido precedente en la implementación de software matemático educativo. El sistema logra un balance notable entre precisión matemática, eficiencia computacional y usabilidad, demostrando que es posible crear herramientas accesibles sin comprometer el rigor académico.

Las dificultades encontradas durante el desarrollo, particularmente en la gestión de casos Edge y la optimización de algoritmos complejos, se transformaron en oportunidades de aprendizaje valiosas. El desafío del Teorema Chino del Resto generalizado, por ejemplo, obligó a un profundo análisis de las condiciones de solubilidad de sistemas de congruencias, ayudando en nuestra comprensión tanto matemática como computacional del problema.

La integración de componentes reveló la importancia de diseñar interfaces bien definidas entre módulos. El sistema de parsing de argumentos, inicialmente una fuente de problemas evolucionó hacia una solución robusta que maneja elegantemente la diversidad de inputs requeridos por las diferentes funciones matemáticas.

Las posibles mejoras identificadas representan no solo oportunidades técnicas sino también direcciones pedagógicas prometedoras. La expansión hacia criptografía básica, por ejemplo, podría transformar IMAT-LAB de una herramienta puramente matemática a una plataforma para enseñanza de seguridad informática, mostrando aplicaciones prácticas directas de la aritmética modular.

El proyecto ha demostrado ser sustentable técnicamente, con una arquitectura que facilita tanto el mantenimiento como la extensión. La organización modular permitiría incorporar nuevas funcionalidades sin afectar el código existente, mientras que el sistema de comandos proporciona una interfaz consistente para futuras expansiones.

El modo dual de operación (interactivo y batch) amplía significativamente el alcance del sistema, haciéndolo adecuado tanto para exploración individual como para evaluación automatizada en contextos académicos. Esta flexibilidad asegura que la herramienta pueda adaptarse a diferentes estilos de aprendizaje y escenarios educativos.

El proyecto consolidó lecciones valiosas en múltiples dimensiones del desarrollo de software: desde la importancia del testing riguroso en aplicaciones matemáticas hasta la necesidad de documentación clara en herramientas educativas. También destacó cómo el diseño cuidadoso de interfaces puede hacer que conceptos complejos sean accesibles sin sacrificar precisión ni aumentar los tiempos de ejecución de los algoritmos.

La experiencia demostró que el desarrollo de software científico requiere un balance único entre competencias técnicas y comprensión teórica, y que las mejores implementaciones surgen cuando ambos aspectos se retroalimentan mutuamente.

IMAT-LAB establece un fundamento sólido para futuras investigaciones y desarrollos en software matemático educativo. Su arquitectura y enfoque podrían extenderse a otras áreas de las matemáticas discretas, como teoría de grafos o álgebra abstracta, creando un ecosistema de herramientas educativas integradas

El éxito del proyecto sugiere que hay un espacio valioso para herramientas que combinen rigor matemático con interfaces accesibles, particularmente en educación superior donde los estudiantes deben y pueden transicionar de usuarios de matemáticas a creadores de soluciones matemáticas.

En conclusión, IMAT-LAB no solo cumple con su propósito inicial, sino que demuestra el potencial del software bien diseñado para transformar la experiencia de aprendizaje matemático, uniendo efectivamente el mundo abstracto de la teoría con el concreto de la implementación computacional.

AUTORÍA

- Claudia Maria Lopez Bombin
 - Desarrollo íntegro y total del código de imatlab.py y modular.py,
 - Ejecución de tests de código,
 - Control de tiempos mediante becnhmarks,
 - Documentación de algoritmos y teoremas para su posterior uso, y
 - Redacción de la memoria del proyecto.
- Lucia Lozano Isac
 - Asistencia en el desarrollo de código, y
 - Redacción de la memoria del proyecto.

BIBLIOGRAFÍA

- Fenollosa, A. (14 de January de 2023). *Patrón de diseño Memoization*. Obtenido de Andros Fenollosa:
<https://andros.dev/blog/310972db/patron-de-diseno-memoization/>
- Garcia-Pelayo, G. (15 de 05 de 2012). *Eureka... Los números primos*. Obtenido de Gonzalo Garcia-Pelayo:
<https://gonzalogarciapelayo.com/eureka-los-numeros-primos/>
- Gauss, C. F. (1801). *Disquisitiones Arithmeticae*. Yale University Press.
- Hardy, G. H., & Wright, E. M. (1938). *An Introduction to the Theory of numbers*. Clarendon Press.
- Knuth, D. E. (1968). *The Art of Computer Programming*. Addison-Wesley.
- Nápoli, P. L. (2020). *Departamento de Matemática - Facultad de Ciencias Exactas y Naturales*. Obtenido de Universidad de Buenos Aires: <https://mate.dm.uba.ar/~pdenapo/apuntes-algebra/2020/2do-cuatrimestre/clase-14-Euclides.pdf>
- Palacios, E. M. (2024). *Algebra Basica*. Obtenido de Web Universidad de Granada:
<https://www.ugr.es/~bullejos/Algl/enteros.pdf>
- Prego, J. M. (7 de November de 2009). *El teorema de Euler*. Obtenido de Web Universidad de Granada:
<https://www.ugr.es/~olimpiada/material/congruencias.pdf>